FractalFlow: Portable Deep-Zoom Julia Rendering with Perturbation Theory across WebGPU and CUDA

Timofei Amosov

Technical Report, Version 1.1.0 – 28 July 2026

Abstract

Interactive rendering of a Julia set becomes a numerical problem long before it becomes a throughput problem: direct single-precision iteration can no longer distinguish adjacent pixels once the view width is smaller than the spacing of the represented coordinates. FractalFlow addresses this problem with a high-precision reference orbit computed once per view and low-precision per-pixel perturbations evaluated in parallel. The same mathematical recurrence is implemented in a WebGPU/WGSL fragment shader with 32-bit deltas and in a native CUDA kernel with 64-bit deltas; a WebGL2 direct-iteration backend is kept as a compatibility fallback. The browser stores its view centre and reference orbit in double-double arithmetic, but uploads orbit samples relative to the initial reference point. This avoids a cancellation failure in the rebasing step that otherwise quantizes small offsets on the grid of order-one single-precision values. Four fixed views are compared against independent direct iteration with `mpmath`, and the resulting images, metrics and checksums are archived with the report. On one RTX 3060 Ti system, the measured peak upper-bound throughput is approximately 1,168 GIter/s for WebGPU and 76.6 GIter/s for CUDA, versus 0.023 GIter/s for a vectorized NumPy baseline. These figures are not general hardware rankings: they expose the precision-throughput trade made by each backend. The report documents the algorithm, the implementation-specific precision correction, the limits of the evidence and a reproducible release procedure.

1 Motivation and contribution

For the quadratic map

$$f_c(z) = z^2 + c, \qquad z, c \in \mathbb{C}, \tag{1}$$

the filled Julia set contains the initial values whose forward orbit remains bounded; its boundary is the Julia set [1]. An escape-time image evaluates this orbit for every pixel. At ordinary scales, the method maps well to a GPU fragment shader. At deep scales, however, adding a pixel displacement to an order-one centre can round back to the centre itself. More iterations or more GPU cores cannot recover information that was lost in coordinate representation.

Perturbation rendering separates the precision-sensitive and throughput-sensitive work. A reference orbit is evaluated at high precision once, while neighboring pixels advance only a small deviation from that orbit [2, 3]. FractalFlow applies this established idea specifically to an interactive Julia viewer and makes four engineering contributions:

1. a common Julia perturbation recurrence is implemented in a browser WebGPU shader and a native CUDA kernel;
2. the browser retains a double-double camera and reference orbit while performing the per-pixel loop entirely in `f32`;
3. the reference samples used during rebasing are stored relative to the initial reference point, preventing order-one `f32` quantization from destroying deep offsets; and
4. correctness and throughput evidence are generated by independent scripts and frozen as release data rather than inferred from visual inspection.

The report does not claim to introduce perturbation, double-double arithmetic or rebasing. Its focus is the portable implementation, the numerical failure found at the interface between these techniques, and an evidence package that can be checked independently.

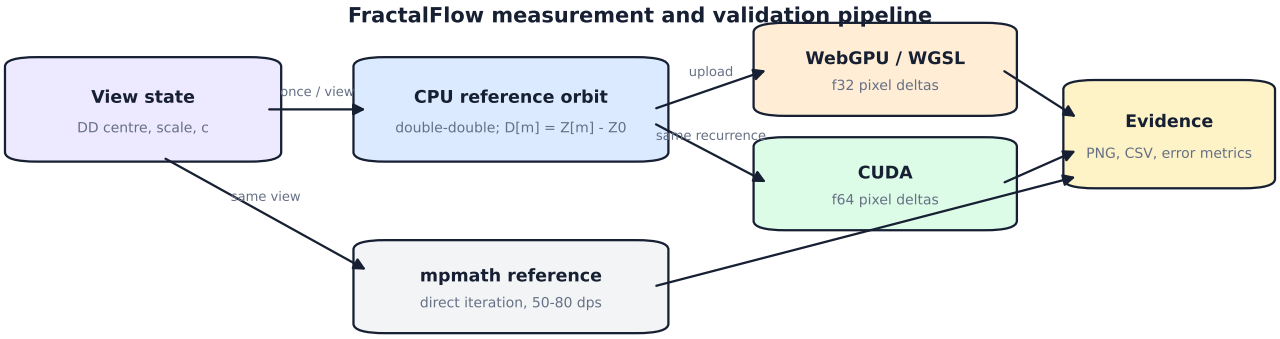

Figure 1: The precision-sensitive reference is computed once on the CPU. WebGPU and CUDA then evaluate the same per-pixel recurrence at different precisions. Direct arbitrary-precision iteration supplies an independent reference rather than reusing the implementation under test.

2 Perturbation for a Julia map

2.1 Reference and delta recurrence

Let Z_n be the reference orbit beginning at the view centre Z_0:

$$Z_{n+1} = Z_n^2 + c. \tag{2}$$

A pixel begins at $z_0 = Z_0 + \delta_0$, where δ_0 is its screen-space offset after scale and aspect-ratio mapping. Writing $z_n = Z_n + \delta_n$ and substituting in Equation 1 gives

$$\begin{aligned} Z_{n+1} + \delta_{n+1} &= (Z_n + \delta_n)^2 + c, \\ \delta_{n+1} &= 2Z_n\delta_n + \delta_n^2. \end{aligned} \tag{3}$$

Unlike the corresponding Mandelbrot derivation, there is no δc term: c is constant across a Julia image and the initial value varies per pixel. Once Z_n is known, Equation 3 uses only ordinary complex products and additions.

FractalFlow stores at most 4000 reference points. Each pixel maintains a reference index m, an iteration count and the complex delta. The escape test uses $z = Z_m + \delta$; escaped pixels terminate early, while interior pixels stop at the iteration limit. Smooth coloring is applied only after the escape decision and is kept identical between the measured renderers and the Python reference.

2.2 Rebasing

A delta can cease to be small relative to its reference or the finite reference orbit can run out. FractalFlow adapts the rebasing strategy credited to Zhuoran and documented by Heiland-Allen [4]. After a delta advance, the implementation forms the pixel displacement from the initial reference point,

$$\delta^{(0)} = z - Z_0, \tag{4}$$

and resets m to zero when $|\delta^{(0)}| < |\delta|$ or the next reference sample is unavailable. The crucial expression is $z - Z_0$, not z. These coincide only when $Z_0 = 0$, which explains why an incorrect Mandelbrot-style reset can remain hidden at the origin and fail in an off-centre Julia view.

3 Precision design

3.1 Double-double camera and orbit

A double-double number represents a value as the unevaluated sum $x_{\mathrm{hi}} + x_{\mathrm{lo}}$ of two binary64 values. Error-free transformations such as TwoSum, QuickTwoSum and Dekker's split product recover the rounding residual of elementary operations [5–7]. FractalFlow implements only addition, subtraction and multiplication. The representation provides roughly 106 significand bits, or about 31 decimal digits, while retaining the binary64 exponent range.

The camera centre remains double-double through cursor-anchored zoom and pan. The CPU also evaluates Equation 2 in double-double. The browser does not emulate this arithmetic per pixel: doing so would replace a small number of CPU operations with millions of compensated shader operations. Instead, the reference is rounded once for GPU upload and the pixel delta remains a native `f32`.

3.2 Why the uploaded orbit is relative

An early implementation uploaded absolute orbit samples $\mathrm{fl}_{32}(Z_m)$. This is sufficient for multiplying the small delta by an approximate order-one Z_m, but it is unsafe for Equation 4. Forming $\mathrm{fl}_{32}(Z_m)+\delta-Z_0$ subtracts nearby order-one values after they have already been rounded to a grid whose spacing is approximately 10^{-7}. At a view scale of 10^{-10}, many distinct pixel offsets therefore collapse to zero or to the same grid point.

The current representation uploads

$$D_m = \mathrm{fl}_{32}(Z_m - Z_0), \tag{5}$$

where the subtraction is performed in double-double on the CPU. The rebased delta is then $D_m + \delta$: both terms are small precisely when their sum requires relative precision. The shader still reconstructs an approximate Z_m for the recurrence as $\mathrm{fl}_{32}(Z_0)+D_m$, but the precision-critical subtraction is no longer performed between two rounded order-one samples.

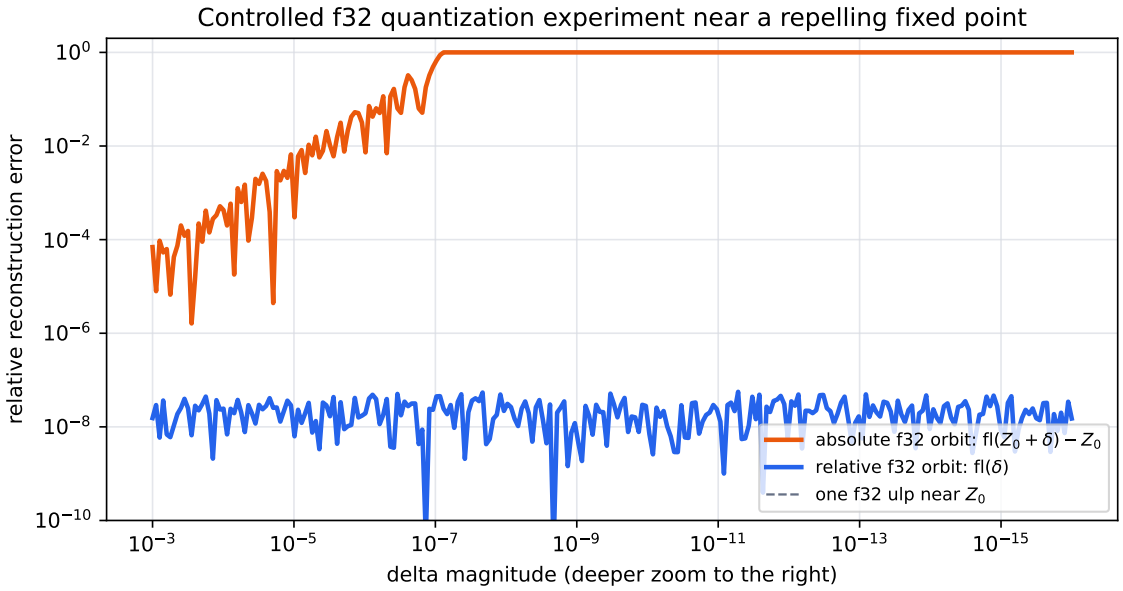

Figure 2: Controlled binary32 experiment at the fixed point used in the validation suite. Encoding the absolute coordinate loses the delta once it falls below the local unit in the last place. Encoding the offset keeps an approximately constant relative error. This plot illustrates the quantization mechanism; it is not a renderer benchmark.

4 Three execution backends

Figure 1 summarizes the shared pipeline. The TypeScript host owns the camera, computes the reference and selects a renderer through a small common interface.

WebGPU. The primary browser backend uses a WGSL fragment shader and a storage buffer for the relative reference orbit. WebGPU maps browser workloads to modern native GPU APIs [8]; WGSL defines the portable shader language and its host-shareable data layout [9]. Every fragment evaluates Equation 3 in `f32`. A cached reference orbit is reused until the view, Julia parameter or iteration limit changes.

CUDA. The native executable repeats the host-side double-double orbit calculation and launches a CUDA kernel whose deltas are binary64. CUDA provides native single- and double-precision arithmetic, but their throughput can differ substantially by device class [10]. The executable supports both PNG rendering and a benchmark schedule identical to the browser harness.

WebGL2 fallback. The compatibility backend uses a double-single GLSL coordinate representation and direct iteration. On the tested Windows ANGLE/D3D11 path, the compensated arithmetic is not stable at the deepest scales; it is therefore neither used as validation evidence nor included in the throughput chart. This exclusion is important: the speed of a numerically wrong image is not a useful performance result.

5 Correctness protocol

5.1 Independent reference

The validation path performs direct iteration of Equation 1 with `mpmath` at 50–80 decimal digits [11]. It contains no reference orbit, perturbation or rebasing logic. Candidate and reference renders share only the declared pixel mapping, escape radius, iteration limit and color function. This independence is more useful than comparing the WGSL and CUDA ports with each other because a shared algebraic error could make two ports agree.

Four 64×64 cases exercise distinct failure modes: an overview, an off-centre boundary, a 10^{-5} deep view and a 10^{-10} view centred on a repelling fixed point of $z^2 - 0.8 + 0.156i$. The script records the mean absolute RGB error, maximum channel error, percentage of identical pixels, percentage whose three channels are all within four levels, and SHA-256 hashes of both PNG files. A mean error of three levels is the release failure threshold; the exact observations below are read from the archived CSV.

Table 1: CUDA perturbation output versus direct `mpmath` rendering. Error units are 8-bit RGB levels, averaged over channels and pixels.

Case	Scale	Mean error	Identical (%)	Within 4 (%)
Overview	3	0.000,163	99.95	100.00
Off-centre boundary	3×10^{-1}	0.098,796	99.54	99.83
Deep zoom	1×10^{-5}	0.000,163	99.95	100.00
Repelling fixed point	1×10^{-10}	0.005,941	99.46	99.98

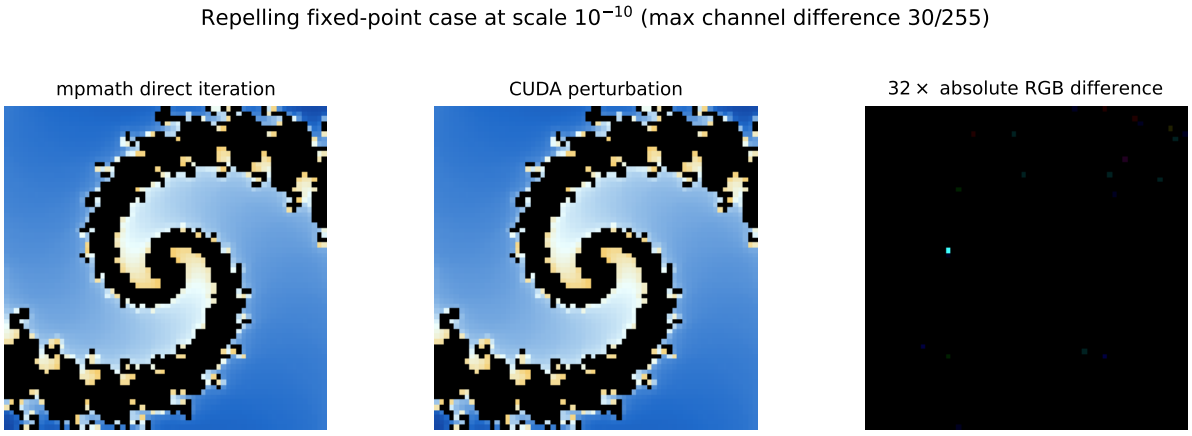

Figure 3: The most precision-sensitive fixed case. The third panel amplifies absolute channel differences by a factor of 32; the unamplified maximum is printed in the figure title by the generation script.

The frozen pixel-level matrix covers CUDA, not WebGPU. The browser path shares the tested TypeScript double-double and relative-orbit preparation, but advances the delta in binary32 and is currently guarded only by unit tests plus the benchmark luma readback. This is an explicit evidence gap: a future release should export raw WebGPU pixels under automation and compare them with the same `mpmath` reference. The current report therefore does not present the CUDA image agreement as a universal proof for every backend.

6 Performance experiment

6.1 Method

The benchmark uses 13 view scales $3{\times}10^{-k}$, $k = 0, \ldots, 12$, at 1920×1080. The iteration cap is $\min(300{+}800k, 4000)$. For each depth, the browser reports the median of five timed frames after three warm-up frames. Completion is synchronized with `GPUQueue.onSubmittedWorkDone`; rendering targets an off-screen texture, and a luma readback rejects silently black output. CUDA uses events around the kernel. The reference orbit is prepared and cached before the timed region.

The reported iteration rate is

$$R = \frac{WHI_{\max}}{t}. \tag{6}$$

This is an upper bound because escaped pixels terminate before $I_{\max}$. It remains useful for a controlled backend comparison because the schedule, view, escape behavior and counting convention are shared. The NumPy baseline [12] times vectorized direct binary64 iteration for the shallow anchor only and excludes coloring. The timed GPU kernels include their palette evaluation and target writes; luma readback and file encoding remain outside the timed region.

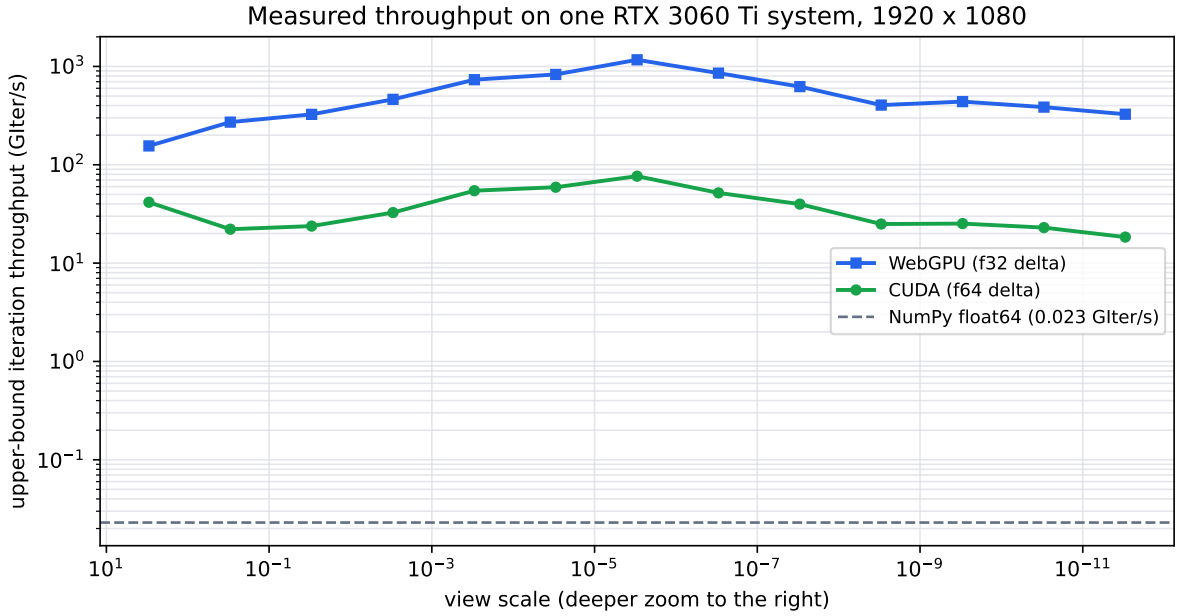

Figure 4: Frozen release measurements. The curves describe one software and hardware configuration, not WebGPU or CUDA in general. GIter/s uses the upper-bound convention in Equation 6.

Table 2: Peak upper-bound GIter/s extracted from the archived CSV files, with Mpix/s from the same row. The CPU values refer to its single shallow anchor.

Backend	Peak GIter/s	Mpix/s	Scale
WebGPU f32	1,168.225	292.1	3×10^{-6}
CUDA f64	76.547	19.1	3×10^{-6}
NumPy f64	0.023	0.1	3

6.2 Interpretation

The observed WebGPU peak is approximately 15.3 times the CUDA peak and roughly 51,000 times the NumPy GIter/s anchor. This does not imply that a browser is intrinsically faster than native CUDA. The two GPU kernels execute different precision choices: WebGPU uses binary32 deltas while CUDA uses binary64. On the tested consumer Ampere device, the extra precision has a large throughput cost. Perturbation makes the binary32 path viable by concentrating extended precision in the single CPU reference orbit.

The curves also vary with depth because early escape, control-flow divergence and the orbit structure change even when $I_{\max}$ has reached its cap. Consequently, a peak alone does not predict interactive frame time for an arbitrary Julia parameter or viewport.

7 Reproducibility and limitations

The release contains source code, lock files, benchmark CSVs, validation PNGs, checksums and a figure-generation manifest. The essential commands are:

```
npm ci
npm test
npm run build
nvcc -O3 cuda/julia.cu -o cuda/julia.exe
uv --cache-dir .uv-cache run python scripts/validate_release.py
cuda/julia.exe --bench --w 1920 --h 1080 \
  --out paper/data/benchmark/cuda_bench.csv
uv --cache-dir .uv-cache run python paper/generate_figures.py
latexmk -cd -norc -pdf paper/main.tex
```

The WebGPU CSV is generated at `/?bench=webgpu`; the page also emits a machine-readable console record. The report PDF is rebuilt in CI with TeX Live 2026. The software release is licensed under Apache-2.0, while this report and its original figures are CC BY 4.0.

The evidence has five main limitations. First, performance is measured on one RTX 3060 Ti system and one browser/driver stack. Second, RGB comparison can hide compensating differences in raw iteration state, although the identical color mapping and fixed cases make large errors visible. Third, one reference orbit and the current rebasing rule are not a proof of correct rendering for every Julia parameter. Fourth, the WebGL2 fallback is a compatibility path, not a validated deep-zoom backend on the tested ANGLE configuration. Finally, double-double limits the current camera to roughly 31 decimal digits; it is not an arbitrary-depth representation.

8 Conclusion

FractalFlow demonstrates a practical division of numerical labor: compute one reference orbit with extended precision, then spend the GPU budget on cheap per-pixel deltas. The portable recurrence is short; the difficult part is preserving the meaning of a small delta at every representation boundary. Storing $Z_m - Z_0$ rather than Z_m makes the rebasing path scale with the view and removes an order-one binary32 quantization failure. Independent arbitrary-precision renders and frozen benchmark inputs turn that observation into a checkable software result. Future work includes raw iteration-state validation, additional devices and browsers, series approximation, progressive tiling and a higher-precision camera beyond 10^{-28}-scale views.

References

[1] John Milnor. *Dynamics in One Complex Variable*. Princeton University Press, 3 edition, 2006. DOI: 10.1515/9781400835539.

[2] K. I. Martin. Superfractalthing maths. https://dirkwhoffmann.github.io/DeepDrill/docs/_downloads/be97cde12ee907c901e6ae8946843d94/sft_maths.pdf, 2013. Technical description of perturbation and series approximation; accessed 2026-07-28.

[3] Claude Heiland-Allen. Perturbation techniques applied to the mandelbrot set. Technical report, mathr, 2013. https://mathr.co.uk/mandelbrot/perturbation.pdf, accessed 2026-07-28.

[4] Claude Heiland-Allen. Deep zoom theory and practice (again). https://www.mathr.co.uk/blog/2022-02-21_deep_zoom_theory_and_practice_again.html, 2022. Discussion of Zhuoran's rebasing method; accessed 2026-07-28.

[5] Theodorus J. Dekker. A floating-point technique for extending the available precision. *Numerische Mathematik*, 18:224–242, 1971. DOI: 10.1007/BF01397083.

[6] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3 edition, 1998.

[7] Yozo Hida, Xiaoye S. Li, and David H. Bailey. Algorithms for quad-double precision floating point arithmetic. Technical Report LBNL-46996, Lawrence Berkeley National Laboratory, 2000. https://escholarship.org/uc/item/69q5t2mj, accessed 2026-07-28.

[8] W3C GPU for the Web Working Group. *WebGPU*. World Wide Web Consortium, 2026. https://www.w3.org/TR/webgpu/, accessed 2026-07-28.

[9] W3C GPU for the Web Working Group. *WebGPU Shading Language*. World Wide Web Consortium, 2026. https://www.w3.org/TR/WGSL/, accessed 2026-07-28.

[10] NVIDIA Corporation. *CUDA C++ Programming Guide, Release 13.0*. NVIDIA Corporation, 2025. https://docs.nvidia.com/cuda/archive/13.0.0/cuda-c-programming-guide/index.html, accessed 2026-07-28.

[11] The mpmath development team. *mpmath: A Python Library for Arbitrary-Precision Floating-Point Arithmetic, Version 1.4.1*, 2026. https://mpmath.org/, accessed 2026-07-28.

[12] Charles R. Harris et al. Array programming with numpy. *Nature*, 585:357–362, 2020. DOI: 10.1038/s41586-020-2649-2.